Tema 4:

Pilas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Sergio Cavero

Universidad
Rey Juan Carlos

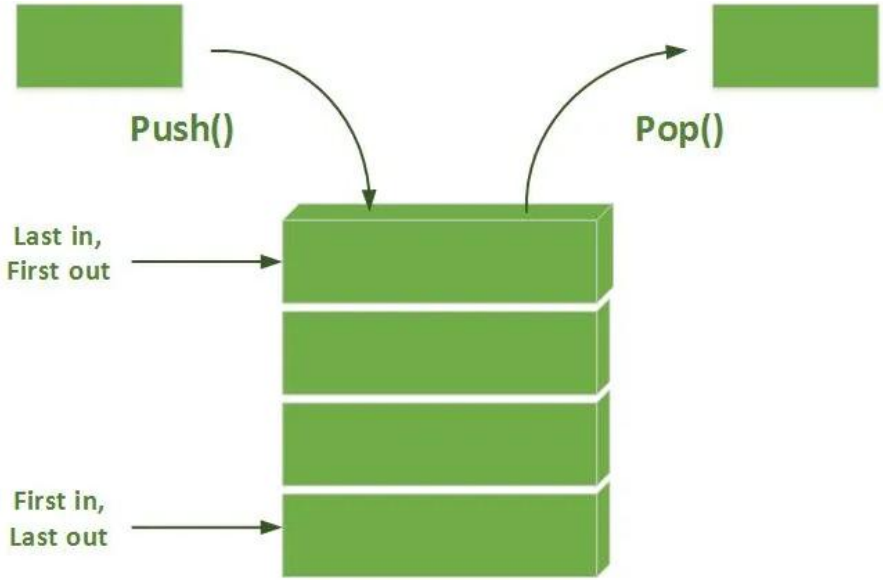
Curso 2025-2026

Contenidos

- Definición
- El TAD Pila: operaciones básicas
- Implementación
- Ejemplos de aplicación
- Anexo: Cómo resolver los ejercicios

Pila: definición

- Estructura de datos lineal, dinámica, que da soporte al concepto **LIFO**: **L**ast **I**n, **F**irst **O**ut

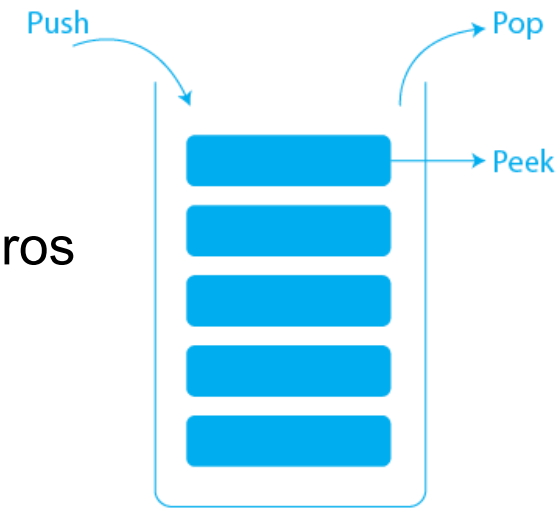


TAD Pila

- Independientemente de su implementación, una pila debe proporcionar al menos 4 operaciones básicas:
  - Meter un elemento (`push`)
  - Sacar un elemento (`pop`)
  - Consultar si está vacía (`is_empty`)
  - Conocer el elemento de la cima (`peek`/`top`)
  - \+ Inicializar/crear

TAD Pila: especificación

- **Nombre:** Pila.
- **Valores posibles:** *vacía, otros (según elementos)*.
- **Operaciones:**
  - `push`
  - `pop`
  - `is_empty`
  - `peek`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

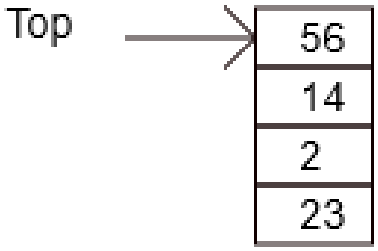


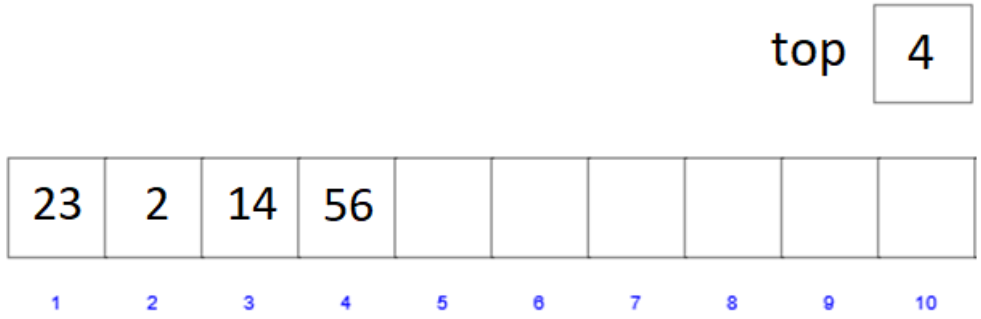
Implementación

- Dos formas habituales de implementarla: con un **array** o utilizando **memoria dinámica**
  - Externamente no debería ser posible distinguirlo
- Utilizando arrays hay limitaciones de espacio, pero la implementación es muy sencilla
- Utilizando punteros y memoria dinámica no hay límite de espacio (en teoría) pero la implementación es algo más compleja

Implementación con un array

- Necesitaremos una variable entera que indicará la posición de la cima + un array del tipo base a almacenar





Implementación con un array

- Utilizaremos un array cuyo índice empiece por 1
- Habrá un máximo de elementos que estableceremos de manera estática (constante MAX_ELEMENTOS)
- Necesitaremos una variable entera para registrar la posición del elemento cima (0 si está vacía)
- Antes de insertar o borrar un elemento comprobaremos si ello es posible (precondiciones)
- Al sacar un elemento, no haremos un borrado físico del mismo en el array, simplemente decrementaremos el índice del elemento cima.
- La inserción y borrado tienen complejidad O(1): muy eficientes.

Implementación con un array

```
unit uPilaEnteros;

interface

const MAX_ELEMENTOS = 100;

type
        tPilaEnteros = record
                cima: integer;
                stack: array[1..MAX_ELEMENTOS] of integer;
        end;

procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;
```

Implementación con un array

```
procedure initialize(var p: tPilaEnteros);
begin
	p.cima := 0;
end;


function isEmpty(p: tPilaEnteros): boolean;
begin
	isEmpty := p.cima = 0;
end;
```

Implementación con un array

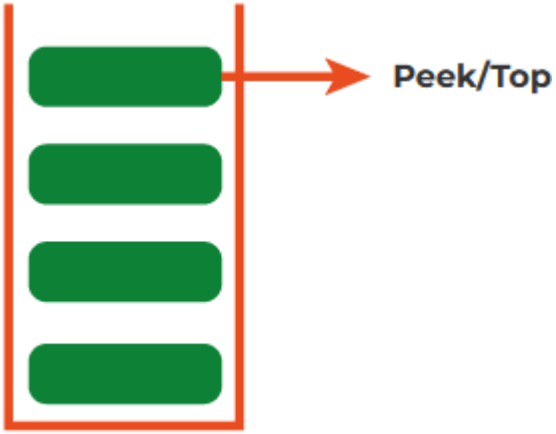
```
procedure push(var p: tPilaEnteros; x: integer);
begin
    if p.cima < MAX_ELEMENTOS then
    begin
        p.cima := p.cima + 1;
        p.stack[p.cima] := x;
    end;
end;
```

Implementación con un array

```
procedure pop(var p: tPilaEnteros);
begin
  if not isEmpty(p) then
      begin
            p.cima := p.cima - 1;
      end;
end;
```

Implementación con un array

```
function peek(p: tPilaEnteros): integer;
begin
    if not isEmpty(p) then
        peek := p.stack[p.cima];
end;
```



Discusión: precondiciones

- Las funciones `pop`, `push`, `peek` están sujetas a que se dé una cierta condición para que su funcionamiento sea el esperado.
  - `push` requiere espacio disponible (evitar *overflow*)
  - `pop`/`peek` necesitan pila no vacía (prevenir *underflow*)
- Es necesario por parte del programador una decisión de implementación para cada una.
  - Por ejemplo, en el caso de push, es el programador el que debe detectar que la pila está llena.
  - En lenguajes que soporten excepciones, lo adecuado es hacer uso de estas.

Funcionalidades adicionales

- Podemos ampliar nuestro TAD Pila con algunas operaciones extra tales como:
  - `is_full()` → Comprueba si está llena
  - `clear()` → Vacía la pila

- Se proponen como ejercicio.

Implementación con un array

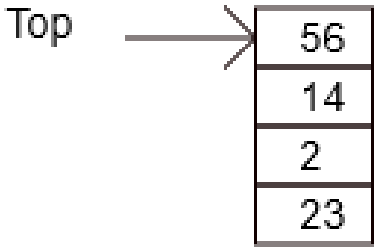
```
{ ------------- Funcionalidades extra --------- }

function isFull(p: tPilaEnteros): boolean;
begin
	isFull := p.cima = MAX_ELEMENTOS;
end;

procedure clear(var p: tPilaEnteros);
begin
	p.cima := 0;
end;
```

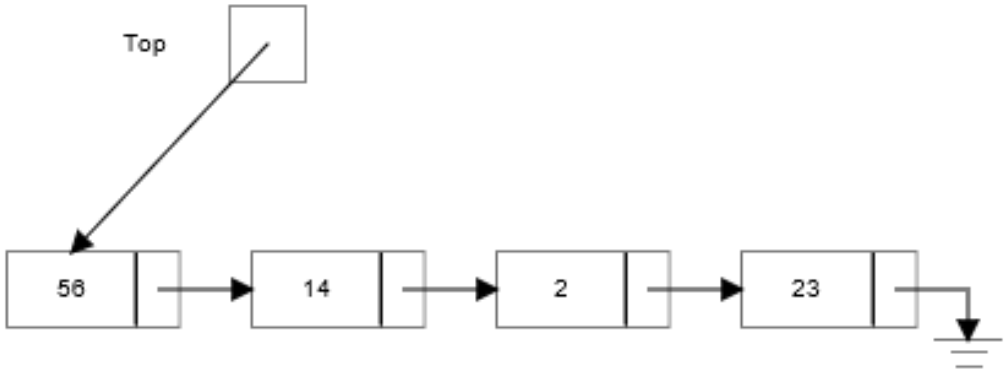
Pila implementada con array (resumen)

- **Desventaja**: es necesario conocer y establecer un tamaño máximo a priori.
- Es necesario establecer **precondiciones** en varias operaciones cuya implementación más lógica sería utilizar excepciones:
  - Pascal no da soporte eficiente a este concepto
  - Las implementaciones propuestas dejan abierta la puerta a comportamientos imprevistos si no se emplean correctamente
  - Es posible una reflexión diferente para cada tipo base

Implementación en memoria dinámica

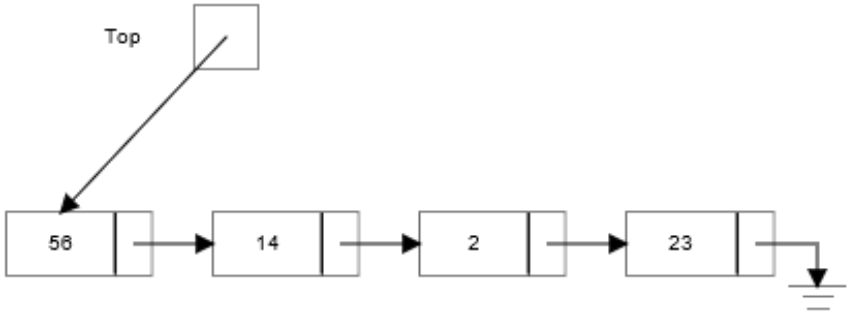
- Necesitaremos un tipo “nodo” para representar cada elemento apilado y un puntero a la cima.





Implementación en memoria dinámica

- Utilizaremos una estructura auxiliar “nodo” con dos partes:
  - la información (del tipo base)
  - un puntero al nodo anterior (el que está “debajo” de él en la pila o `nil` si no hay ninguno más)
- Al insertar un elemento hay que crear un nuevo nodo y poner el puntero de pila a apuntar al mismo, ya que entra en la parte superior
- No hay que comprobar *overflow* pues no tiene límite de almacenamiento, solo *underflow* (`is_empty`)



Implementación en memoria dinámica

```
unit uPilaEnteros;

interface

type
      tNodo = record
            info: integer;
            ant: ^tNodo;
      end;

  tPilaEnteros= ^tNodo; // cima

procedure initialize(var p: tPilaEnteros);
procedure push(var p: tPilaEnteros; x: integer);
procedure pop(var p: tPilaEnteros);
function peek(p: tPilaEnteros): integer;
function isEmpty(p: tPilaEnteros): boolean;
```

Implementación en memoria dinámica

```
implementation

procedure initialize(var p: tPilaEnteros);
begin
    p := nil; { Inicializa el puntero cima de la pila a nil }
end;

function isEmpty(p: tPilaEnteros): boolean;
begin
    isEmpty := p = nil; { Devuelve true si la cima es nil }
end;
```

Implementación en memoria dinámica

```
procedure push(var p: tPilaEnteros; x: integer);
var
    nuevo: ^tnodo;
begin
    new(nuevo);  { Crea un nuevo nodo }
    nuevo^.info := x; { Asigna el valor al nodo }
    nuevo^.ant := p;          { Enlaza el nuevo nodo con la anterior cima }
    p := nuevo;  { Actualiza la cima de la pila }
end;
```

Implementación en memoria dinámica

```
procedure pop(var p: tPilaEnteros);
var
    aux: ^tnodo;
begin
    if not isEmpty(p) then
    begin
        aux := p;        { Apunta con un puntero auxiliar al nodo en la cima (a eliminar) }
        p := p^.ant;     { Actualiza la cima de la pila }
        dispose(aux);    { Libera la memoria del elemento a eliminar }
    end;
end;
```

Implementación en memoria dinámica

```
function peek(p: tPilaEnteros): integer;
begin
    if not isEmpty(p) then
        peek := p^.info; { Devuelve la parte información de la cima de la pila }
end
```

Funcionalidades adicionales

- Podríamos ampliar nuestro TAD Pila con operaciones extra tales como:
  - `clear()` → Vacía la pila
  - `npop(n)` → Saca n elementos de la pila

- Se proponen como ejercicio.

Aplicaciones

- Las pilas son estructuras muy utilizadas en computación en aplicaciones tales como:
  - Evaluación de expresiones
  - Llamadas a subprogramas
  - Guardar listado de órdenes para deshacer-rehacer en editores
  - Adelante-atrás en navegadores Web
  - Algoritmos de Backtracking (ej. Colocar 9 reinas en un tablero de ajedrez, búsqueda de salida en un laberinto, etc.)
  - Gestión de memoria
  - Etc.

ANEXO:

Cómo resolver los ejercicios

¿Cómo enfrentarse a los ejercicios?

1. Lee detenidamente el enunciado completo.
2. Identifica el objetivo del ejercicio:
3. Abre cada uno de los archivos proporcionados y estudia el código proporcionado.
4. Haz que el programa compile y ejecuta el programa aunque no haga lo que se pide. Ya iremos avanzando.
5. Realiza los ejercicios de manera ordenada, comprobando que cada uno de ellos funciona correctamente antes de pasar al siguiente.

Identificar el objetivo de un ejercicio

Posible objetivo 1: Identificar el tipo de elementos que se almacenarán en la pila

1. Ejemplos: enteros, caracteres, registros, etc.
2. Esto va a determinar la definición de los nodos de la pila (o el tipo de array en caso de no ser una pila dinámica).
3. Es decir, deberemos **trabajar en la propia unidad** de la pila. Tendremos el rol de implementador de la unidad.

Identificar el objetivo de un ejercicio

Posible objetivo 2: Ampliar o modificar funcionalidades de una pila ya implementada

1. Ejemplos: contar el número de elementos de la pila, verificar si un elemento está en la pila, eliminar un elemento, etc.
2. Generar nuevos procedimientos o funciones requerirá utilizar las operaciones principales.
3. Es decir, nuevamente deberemos **trabajar en la propia unidad** de la pila ya que estamos extendiendo su funcionalidad. Tendremos de nuevo el rol de implementador de la unidad.

Identificar el objetivo de un ejercicio

Posible objetivo 3: Utilizar una pila para resolver un problema específico

1. Ejemplos: verificar si una expresión aritmética está balanceada, verificar si una palabra es palíndroma, utilizarla para representar un conjunto de elementos, etc.
2. En este caso, deberemos utilizar las operaciones básicas de la unidad de la pila para resolver el problema, pero no podemos modificar la pila añadiendo o cambiando funcionalidades.
3. Por lo tanto, deberemos **usar** la pila en nuestro programa principal para resolver el problema. Tendremos el rol de usuario externo de la unidad pila y por lo tanto solo podemos usar las funcionalidades definidas en la misma.

A tener en cuenta…

- Una pila es una estructura de datos bien definida con esencialmente 5 operaciones básicas: pop, push, isEmpty, peek e initialize.
- Los ejercicios en clase tienen un enfoque principalmente didáctico y por ello podrían solicitarse funcionalidades que van más allá de este comportamiento básico
- Piensa siempre qué rol debes tomar: implementador de la pila o usuario de la pila

implementador de la pila

usuario de la pila

Ejercicios propuestos

Ejercicios propuestos

Implementa las siguientes funciones de la pila:

- `clear()` → Vacía la pila
- `npop(n)` → Saca n elementos de la pila

Antes de empezar, responde las siguientes preguntas:

- ¿Dónde deberían implementarse?
- ¿Qué rol tienes, implementador de la pila o usuario de la pila?

Ejercicios propuestos

Implementa un procedimiento externo a la unidad de la pila que muestre por pantalla todos los elementos que contiene.

Es imprescindible que la pila permanezca exactamente en el mismo estado en que fue recibida tras finalizar la operación.

De nuevo, antes de empezar reflexiona sobre las cuestiones:

- ¿Dónde va a implementarse?
- ¿Qué rol tienes (implementador de la pila o usuario de la pila)?